

DOCUMENTAÇÃO DA ARQUITETURA: PROJETO FINAL - MODERNIZAÇÃO DE UM SISTEMA LEGADO

Autor: Matheus Martins da Cruz

1. ARQUITETURA ESCOLHIDA

A escolha da arquitetura deste projeto foi baseada na utilização de chamadas de arquivos COBOL compilados dinamicamente (.dll) através do atributo [DllImport] da Platform Invocation Services (P/Invoke), que informa que o programa que deve invocar o arquivo COBOL especificado.

São 4 arquivos COBOL desenvolvidos, cada um com apenas uma função, eles são:

- CONSCLI.cbl - Consulta clientes lendo o arquivo CLIENTES.dat (VSAM).
- CADASCLI.cbl - Cadastra clientes no o arquivo CLIENTES.dat (VSAM).
- ATUACLI.cbl - Atualiza clientes no arquivo CLIENTES.dat (VSAM).
- DELECLI.cbl - Deleta clientes no arquivo CLIENTES.dat (VSAM) .

E 2 arquivos VSAM, eles são:

- CLIENTES.dat - Onde estão todos os registros de clientes cadastrados do sistema, garantindo a persistencia, utilizado em todos programas COBOL do projeto.
- CONTROLE.dat - Utilizado apenas no programa de cadastro, serve de auxilio para garantir o auto-incremento de cadastros, sendo criado automaticamente se não existir no projeto contendo o maior ID encontrado no arquivo CLIENTES.dat.

Desta maneira, temos a integração .NET e COBOL rodando dentro do mesmo processo. Os dados trafegam diretamente na memória por referência, garantindo velocidade e otimização para a aplicação. O projeto foi construido visando manter os códigos COBOL simulando as execuções legadas e o .NET recebendo as informações e adaptando para as tecnologias atuais, isso é explicado de maneira mais aprofundada na Documentação de Estrutura Compartilhada.

2. JUSTIFICATIVAS TÉCNICAS

A pesquisa de arquiteturas para esse projeto foi intensa, como o COBOL em ambiente

acadêmico é bastante limitado, pois a grande maioria das infraestruturas necessitam de licenças (como o z/OS Connect), e o simulador de mainframe Tunkey 5(TK5) não se adapta bem com esse tipo de integração, já que foi construído pensando em outro propósito, foi analisado e validado que o compilador GnuCOBOL é o ideal para contexto do projeto.

Após a definição de qual compilador (GnuCOBOL) e ambiente (Virtual Studio Code) seria utilizado no desenvolvimento, entramos em outra pauta, o primeiro teste foi com os arquivos COBOL executáveis, onde havia chamada direta via argumento e utilizava o "System.Diagnostics.Process" para ativar o executável e passava o ID do cliente (como era teste de arquitetura só possuía os campos id, nome e status) como um argumento de linha de comando para o executável e capturava o que o COBOL imprimia na tela. Desta maneira funcionou, porém ficou um sistema pouco confiável e a busca de uma maneira mais persistente foi acionada. Posteriormente a conclusão de usar o Platform Invocation Services (P/Invoke) se mostrou mais condizente.

A persistência de dados também foi uma etapa que teve testes relevantes. O primeiro teste foi com arquivo VSAM (CLIENTES.dat), funcionou como esperado garantindo a persistência do sistema. Com o propósito de deixar o projeto no padrão que o mercado tende a exigir, ocorreu o teste com Banco de Dados (DB2). Os testes foram feitos no ambiente Gix_IDE (utilizado no Projeto 6 do Acelera Maker) já que existe a falta de compatibilidade do pré-compilador para a utilização dos comandos SQL no GnuCOBOL, porém usando DB2 deixou o projeto mais instável devido adaptação de Drivers, mudanças de ambiente com recursos extremamente limitados e assim aparentava "gambiarra" entrando em más práticas de padrão de projeto. Depois de analisar foi decidido manter a utilização de arquivo VSAM para persistir os dados pela confiabilidade que apresenta.

3. FLUXOGRAMAS DE EXECUÇÃO DO PROGRAMA

Observação: *todos os fluxogramas dos endpoints estão na Pasta "Fluxogramas".*

- **Interface** - no Swagger o usuário tem a opção de escolher entre 4 endpoints:
 - GET - Busca cadastro pelo Id
 - POST - Cadastra o usuário com nome (obrigatório), telefone e e-mail.
 - PATCH - Atualiza telefone e e-mail do cadastro, ambos são opcionais.
 - DELETE - Deleta cadastro do usuário do sistema.
- **Controlador** - recebe as requisições do swagger e envia para o endpoint correto tratar, eles são:
 - Get - Recebe o id e passa por parâmetro dentro do "try" para a função Consultar() da classe ClienteService. Se ocorrer tudo certo retorna os dados do cadastro, se não encontrou cliente, retorna "Cliente não encontrado.", caso deu problema no VSAM ou algo inválido, retorna "Erro ao consultar o cliente.", e se o try falhar que dizer que deu problema no sistema, retorna "Ocorreu um erro inesperado no servidor.".

- Post - Recebe o dados do cadastro e passa por parâmetro dentro do "try" para a função Cadastrar() da classe ClienteService. Se ocorrer tudo certo retorna os dados do cadastro, se id do cliente já existe, retorna "Cliente já cadastrado.", caso deu problema no VSAM ou algo inválido, retorna "Não foi possível cadastrar.", e se o try falhar que dizer que deu problema no sistema, retorna "Ocorreu um erro inesperado no servidor."
- Patch - Recebe o id os dados a serem atualizados e passa por parâmetro dentro do "try" para a função Atualizar() da classe ClienteService. Se ocorrer tudo certo retorna os dados do cadastro atualizado, se não encontrou cliente, retorna "Cliente não encontrado.", caso deu problema no VSAM ou algo inválido, retorna "Erro ao atualizar o cliente.", e se o try falhar que dizer que deu problema no sistema, retorna "Ocorreu um erro inesperado no servidor."
- Delete - Recebe o id e passa por parâmetro dentro do "try" para a função Deletar() da classe ClienteService. Se ocorrer tudo certo retorna NoContent(), se não encontrou cliente, retorna "Cliente não encontrado.", caso deu problema no VSAM ou algo inválido, retorna "Erro ao deletar o cliente.", e se o try falhar que dizer que deu problema no sistema, retorna "Ocorreu um erro inesperado no servidor."

- **Service** - Toda a lógica e regras de negócio do sistema

- A base é a mesma para todas, primeira parte chama a função parser() e o caminho do CopyBook (REGCLI.cpy), o parser é detalhado na Documentação de Estrutura Compartilhada. Resumidamente ele traduz os campos do CopyBook para o C# e depois vem wrapper onde vai utilizar os campos traduzidos e popular com os dados da requisição invocada e enviar os dados de maneira correta para COBOL garantindo a integração.
 - Consultar - envia apenas o Id deixando os outros campos vazios e recebendo os dados persistidos no VSAM do COBOL e o retorno do resultado da aplicação(status), passando pelo mapeamento do DTO para o response.
 - Cadastrar - envia apenas os dados recebidos pelo ClienteRequestDTO nome, telefone e e-mail, o id vai ser criado pelo COBOL de maneira automática e o status é para o retorno do COBOL, passando pelo mapeamento do DTO para o response.
 - Atualizar - envia o id e os dados recebidos pelo ClienteUpdateDTO e recebendo os dados atualizados com e o retorno do resultado da aplicação(status), passando pelo mapeamento do DTO para o response.
 - Deletar - envia apenas o Id deixando os outros campos vazios e o retorno do resultado da aplicação(status), .

4. COMPONENTES UTILIZADOS

Árvore estrutural do projeto:

```

CodigoFonte/
├── dotnet/
│   ├── Controllers/
│   │   └── ClientesController.cs
│   ├── Services/
│   │   ├── IClienteService.cs
│   │   └── ClienteService.cs
│   ├── Infrastructure/
│   │   ├── CopybookParser.cs
│   │   └── CopybookWrapper.cs
│   ├── Config/
│   │   ├── DllConfig.cs
│   │   └── DllConfigTests.cs
│   ├── DTO/
│   │   ├── Mappings/
│   │   │   └── ClientesDTOMappings.cs
│   │   └── Clientes/
│   │       ├── ClientesRequestDTO.cs
│   │       ├── ClientesResponseDTO.cs
│   │       └── ClientesUpdateDTO.cs
└── cobol/
    ├── SOURCE/
    │   ├── CONSCLI.cb1/.dll
    │   ├── ATUACLI.cb1/.dll
    │   ├── DELECLI.cb1/.dll
    │   └── CADASCLI.cb1/.dll
```

```
|   |   | DATA/
|   |   |   | CLIENTES.dat
|   |   |   | CONTROLE.dat
|   |   | DATA/
|   |   |   | CLITEMPLATE.dat
|   |   |   | CLITESTS.dat
|   |   | COPYLIB/
|   |   |   | REGCLI.cpy
|   |   |   | FDCLI.cpy
|   | tests/
|   |   | ClienteServiceTests.cs
|   |   | CopyBookTests.cs
```

- **Camada dotnet** - é onde ocorre o gerenciamento do projeto, recebe as requisições pelo Swagger, aplica as validações usando DTOs organiza as rotas REST (Controllers).
- **Camada Integração** - utiliza o atributo [DllImport] e para fazer a tradução dos campos para enviar e receber dados do COBOL (CopybookParser e CopybookWrapper).
- **Camada Legado e Persistência** - os arquivos compilados como dinâmicos (.dll). Regras de negócio que realizam o acesso aos arquivos VSAM indexados (.dat), simulando um banco relacional/VSAM de um mainframe.
- **Camada Governança** - testes automatizados (unitários e de integração) no XUnit que testa a confiabilidade do projeto contra dados inválidos e fluxo resultando como o esperado, com o CI/CD (GitHub Actions) aplicado.